

C++ 智能指针详解

C++ 中的智能指针（`std::unique_ptr`，`std::shared_ptr` 和 `std::weak_ptr`）是为了自动管理动态分配的内存，防止内存泄漏、悬挂指针等问题。这些智能指针属于 C++11 标准，位于 `<memory>` 头文件中。

目录

- [std::unique_ptr](#)
 - [主要方法](#)
 - [注意事项](#)
 - [高效使用建议](#)
- [std::shared_ptr](#)
 - [主要方法](#)
 - [注意事项](#)
 - [高效使用建议](#)
- [std::weak_ptr](#)
 - [主要方法](#)
 - [注意事项](#)
 - [高效使用建议](#)

std::unique_ptr

`std::unique_ptr` 是一种独占所有权的智能指针，表示它唯一拥有一个动态分配的对象。当 `unique_ptr` 被销毁时，所管理的对象会被自动删除。

主要方法

- `get()`：返回原始指针。注意，返回的指针不是智能指针，使用时需小心，避免悬挂指针。

```
std::unique_ptr<int> ptr = std::make_unique<int>(10);
int* raw_ptr = ptr.get();
```
- `release()`：释放当前的所有权，返回原始指针并将 `unique_ptr` 设置为空指针。需要手动管理返回的原始指针。

```
std::unique_ptr<int> ptr = std::make_unique<int>(10);
int* raw_ptr = ptr.release();
```
- `reset()`：重置 `unique_ptr`，释放当前管理的对象，并可以选择性地指向一个新对象。

```
ptr.reset(); // 删除对象并置空
ptr.reset(new int(20)); // 删除当前对象，管理一个新的对象
```
- `std::move()`：用于转移所有权到另一个 `unique_ptr`。一旦转移，原指针变为空指针。

```
std::unique_ptr<int> ptr1 = std::make_unique<int>(10);
std::unique_ptr<int> ptr2 = std::move(ptr1); // ptr1 现在为空指针
```

注意事项

- **不能拷贝**：`std::unique_ptr` 不支持拷贝构造和拷贝赋值，因为它不允许两个 `unique_ptr` 拥有相同资源。
- **所有权转移**：使用 `std::move()` 显式转移所有权，不能通过普通赋值进行拷贝。
- **避免悬挂指针**：使用 `get()` 获取原始指针时要小心，避免在对象销毁后继续使用它。

高效使用建议

- **返回动态创建的对象**：当函数需要返回一个动态分配的对象时，使用 `std::unique_ptr` 可以确保对象生命周期的自动管理。

```
std::unique_ptr<MyClass> createObject() {  
    return std::make_unique<MyClass>();  
}
```

std::shared_ptr

`std::shared_ptr` 是一种共享所有权的智能指针，允许多个 `shared_ptr` 共同拥有一个对象。引用计数为 0 时，才会销毁对象。

主要方法

- `use_count()`：返回当前对象的引用计数。

```
std::shared_ptr<int> ptr1 = std::make_shared<int>(10);  
std::cout << "Use count: " << ptr1.use_count() << std::endl; // 输出 1
```

- `get()`：返回原始指针，方便传递给其他函数，但使用时要小心，避免悬挂指针。

```
int* raw_ptr = ptr1.get();
```

- `reset()`：类似 `std::unique_ptr` 的 `reset`，用于重置当前 `shared_ptr`，释放原有资源，选填新的对象。

```
ptr1.reset(); // 释放资源并置空
```

- `swap()`：交换两个 `shared_ptr` 的内部指针和引用计数。

```
std::shared_ptr<int> ptr2 = std::make_shared<int>(20);  
ptr1.swap(ptr2); // 交换 ptr1 和 ptr2 的内容
```

注意事项

- **循环引用**：`shared_ptr` 使用引用计数，如果两个 `shared_ptr` 相互引用，且没有使用 `weak_ptr`，则可能导致引用计数永远不为零，导致内存泄漏。
 - 解决办法：使用 `std::weak_ptr` 打破循环引用。
- **线程安全**：`shared_ptr` 在更新引用计数时是线程安全的，但多个线程不能同时访问同一个 `shared_ptr` 对象。

高效使用建议

- **避免不必要的共享**：如果不需要共享所有权，使用 `std::unique_ptr` 更为高效，因为 `shared_ptr` 的引用计数机制有额外开销。
- **避免频繁的 `use_count()` 调用**：虽然 `use_count()` 可以查询当前引用计数，但不应该频繁调用它，因为它会带来性能开销。

std::weak_ptr

`std::weak_ptr` 是一种不拥有对象的智能指针，主要用于观察 `shared_ptr` 管理的对象。它不增加对象的引用计数，避免循环引用的问题。

主要方法

- `lock()`：尝试将 `weak_ptr` 转换为 `shared_ptr`，如果对象还存在（即引用计数大于零），返回一个有效的 `shared_ptr`，否则返回一个空的 `shared_ptr`。

```
std::weak_ptr<int> weakPtr = ptr1;
std::shared_ptr<int> lockedPtr = weakPtr.lock();
if (lockedPtr) {
    lockedPtr->greet(); // 使用对象
} else {
    std::cout << "Object has been destroyed\n";
}
```

- **expired()**：检查 `weak_ptr` 指向的对象是否已销毁。如果对象已被销毁，返回 `true`。

```
if (weakPtr.expired()) {
    std::cout << "Object is expired\n";
}
```

- **use_count()**：返回 `weak_ptr` 所关联的对象的引用计数，注意这不增加引用计数。

```
std::cout << "Use count: " << weakPtr.use_count() << "\n";
```

注意事项

- **不影响引用计数**：`weak_ptr` 不增加引用计数，使用时需要通过 `lock()` 获取有效的 `shared_ptr` 才能访问对象。
- **防止循环引用**：`weak_ptr` 常用于打破 `shared_ptr` 之间的循环引用，避免内存泄漏。

高效使用建议

- **与 `shared_ptr` 配合使用**：当你需要在多个地方观察对象的生命周期时，使用 `weak_ptr` 来避免不必要的引用计数增加。

```
std::shared_ptr<MyClass> sharedPtr = std::make_shared<MyClass>();
std::weak_ptr<MyClass> weakPtr(sharedPtr); // weak_ptr 观察 shared_ptr
```

- **缓存和观察**：`weak_ptr` 适合用于缓存管理或事件监听，避免在不需要时延长对象生命周期。

智能指针组合使用建议

1. **避免循环引用**：在涉及多个对象互相持有 `shared_ptr` 时，使用 `std::weak_ptr` 来打破循环引用，避免内存泄漏。

```
class A;
class B {
public:
    std::weak_ptr<A> a;
};
class A {
public:
    std::shared_ptr<B> b;
};
```

2. **多个所有者共享资源**：当多个所有者需要共享资源时，使用 `std::shared_ptr` 来管理资源的生命周期；当某个所有者不再关心对象时，使用 `reset()` 或 `shared_ptr` 的析构函数自动释放对象。
3. **对象生命周期管理**：`std::unique_ptr` 适用于对象有单一所有者的场景，避免内存泄漏。`std::shared_ptr` 和 `std::weak_ptr` 可以在多个地方共享和观察对象，确保资源的有效管理。

通过合理地使用这三种智能指针，可以有效地管理资源、避免内存泄漏和悬挂指针问题，确保程序在复杂的对象生命周期管理中保持高效和稳定。

std::make_unique 和 std::make_shared 介绍

1. std::make_unique

std::make_unique 用于创建一个 std::unique_ptr，并返回指向新分配对象的智能指针。它的语法如下：

```
std::unique_ptr<T> make_unique(Args&&... args);
```

- T 是要创建的对象类型。
- Args&&... args 是要传递给 T 的构造函数的参数。

示例：

```
#include <memory>
#include <iostream>

class MyClass {
public:
    MyClass(int x) : value(x) {
        std::cout << "MyClass constructor called with value " << x << "\n";
    }

    void showValue() const {
        std::cout << "Value: " << value << "\n";
    }

private:
    int value;
};

int main() {
    auto ptr = std::make_unique<MyClass>(10); // 自动调用构造函数
    ptr->showValue();
    return 0;
}
```

2. std::make_shared

std::make_shared 用于创建一个 std::shared_ptr，并返回指向新分配对象的智能指针。它的语法如下：

```
std::shared_ptr<T> make_shared(Args&&... args);
```

- T 是要创建的对象类型。
- Args&&... args 是要传递给 T 的构造函数的参数。

示例：

```

#include <memory>
#include <iostream>

class MyClass {
public:
    MyClass(int x) : value(x) {
        std::cout << "MyClass constructor called with value " << x << "\n";
    }

    void showValue() const {
        std::cout << "Value: " << value << "\n";
    }

private:
    int value;
};

int main() {
    auto ptr = std::make_shared<MyClass>(10); // 自动调用构造函数
    ptr->showValue();
    return 0;
}

```

make_unique 和 make_shared 与手动 new 后使用智能指针的区别

1. 简洁性和安全性

std::make_unique 和 std::make_shared :

- **避免显式使用 new** : 它们简化了智能指针的创建过程, 避免了直接使用 new。这不仅使代码更简洁, 还能降低出错的机会。

```

auto ptr1 = std::make_unique<MyClass>(10); // 推荐方式
auto ptr2 = std::make_shared<MyClass>(10); // 推荐方式

```

- **异常安全**: 如果在 new 后发生异常, new 分配的内存会泄漏。而使用 make_unique 或 make_shared 创建的智能指针, 在构造失败时 (例如构造函数抛出异常) 会自动释放内存, 避免了内存泄漏。

```

// 错误示例, 手动使用 new 可能导致内存泄漏
MyClass* rawPtr = new MyClass(10); // 如果构造函数抛出异常, 内存会泄漏
std::unique_ptr<MyClass> ptr(rawPtr);

```

与此不同, 使用 std::make_unique 下面的代码更加安全:

```

auto ptr = std::make_unique<MyClass>(10); // 如果构造抛出异常, 不会泄漏内存

```

2. 性能

- **std::make_shared** 优于直接使用 new 和 shared_ptr, 因为它在创建 shared_ptr 时, 智能指针和对象的内存是在一次内存分配中完成的。即 shared_ptr 和对象共享同一块内存, 减少了内存分配的开销。

```

auto ptr = std::make_shared<MyClass>(10); // 更高效

```

如果手动使用 new 和 shared_ptr :

```

MyClass* rawPtr = new MyClass(10);
std::shared_ptr<MyClass> ptr(rawPtr); // 需要额外的内存分配

```

在手动创建 shared_ptr 的时候, 内存分配是分开进行的: 一块用于对象, 另一块用于引用计数和其他元数据。这会引入额外的开销。

3. 类的访问权限和控制

`std::make_unique` 和 `std::make_shared` 在创建智能指针时有一些优势，尤其是在控制对象的访问权限上：

- **访问权限控制**：在类设计时，`make_unique` 和 `make_shared` 可以帮助你控制对象的访问权限。你可以将构造函数设置为 `private` 或 `protected`，从而避免直接使用 `new` 来创建对象。

通过 `std::make_unique` 和 `std::make_shared`，你可以在工厂函数中隐藏对象的构造，确保它只能通过特定的方式创建，进一步加强封装性。

```
class MyClass {
private:
    MyClass(int x) : value(x) {}

public:
    static std::unique_ptr<MyClass> create(int x) {
        return std::make_unique<MyClass>(x); // 只能通过此静态方法创建
    }

    void showValue() const {
        std::cout << "Value: " << value << "\n";
    }

private:
    int value;
};

int main() {
    // MyClass obj(10); // 错误，构造函数是私有的
    auto ptr = MyClass::create(10); // 通过工厂方法创建
    ptr->showValue();
    return 0;
}
```

如果直接使用 `new`，则不能控制对象的创建过程，因为 `new` 是一种外部的内存分配方式，不受类的成员控制。

4. 与其他资源管理方式的兼容性

`std::make_unique` 和 `std::make_shared` 作为现代 C++ 的内存管理方式，通常比直接使用 `new` 更容易与其他资源管理方式（如 RAII）结合。尤其是：

- **`std::make_shared` 和线程安全**：`std::shared_ptr` 提供了线程安全的引用计数，确保在多线程环境下对对象的共享访问不会出错。而手动使用 `new` 时，可能需要额外的同步机制。
- **与容器的兼容性**：例如，在使用容器存储智能指针时，`std::make_shared` 可以避免多次内存分配，减少性能开销。

```
std::vector<std::shared_ptr<MyClass>> v;
v.push_back(std::make_shared<MyClass>(10)); // 优化的内存管理
```

总结

1. **`std::make_unique` 和 `std::make_shared`** 提供了更安全、简洁的智能指针创建方式，避免了直接使用 `new` 可能带来的内存泄漏和异常处理问题。
2. **性能**：`std::make_shared` 更高效，因为它减少了内存分配的开销，智能指针和对象共享内存。
3. **类的访问控制**：`make_unique` 和 `make_shared` 有助于加强封装，确保对象的创建过程受到控制，可以避免直接使用 `new`。
4. **异常安全和资源管理**：`std::make_unique` 和 `std::make_shared` 提供更好的异常安全性，并且与 RAII 结合得更好。

在实际编程中，优先使用 `std::make_unique` 和 `std::make_shared`，除非有特殊的需求，否则避免手动使用 `new`，尤其是当对象的生命周期由智能指针管理时。